

InterviewForge AI

Complete Project Roadmap — 8-Week Plan, 3-Member Team, Zero Budget

SPP Problem Statement 7: AI Interview Practice Platform

Team size: 3 members | Duration: 8 weeks | Budget: \$0 (free-tier services only)

1. Project Overview

1.1 The Problem

Every placement season, large numbers of academically strong students underperform in interviews not because they lack knowledge, but because they lack structured, repeated practice with real feedback. Traditional preparation — static question-bank PDFs, YouTube playlists, and occasional peer or placement-cell mock interviews — is unscalable, inconsistent, and non-iterative: a placement cell with limited staff simply cannot give every student more than one or two mock rounds before real interviews begin.

1.2 Target Users

- Final-year and pre-final-year engineering/management students preparing for campus placements
- Students without access to expensive paid mock-interview platforms or well-connected alumni networks
- College placement cells looking to scale practice capacity beyond what human volunteers can offer

1.3 Core Value Proposition

InterviewForge AI replaces one-size-fits-all preparation with an AI interviewer that reads a student's actual resume, asks questions tailored to their specific skills and projects, evaluates each response consistently (removing human mood/bias/bandwidth as a bottleneck), and gives a structured report the student can act on — repeatable as many times as needed, at zero cost.

2. Feature Categorization

Features are split into three tiers so scope stays realistic for a 3-person, 8-week, zero-budget build. Nothing is silently dropped — deferred features are explicitly future work.

Category	Features
MVP — Must-Have	User registration & login (JWT auth); resume upload (PDF/DOCX) with AI parsing into structured skills/projects/education; Technical & HR interview modes (text-based); AI-generated, resume-aware interview questions; AI evaluation of each response (correctness, communication, structure); end-of-session AI report (strengths, weaknesses, improvement roadmap); score history and trend dashboard
Best-Effort / Stretch	Voice mode using the browser's Web Speech API for spoken responses; adaptive follow-up questions based on the previous answer; basic skill-radar chart in the report
Future Work — Deferred	Live coding round with an embedded code editor; company-specific question profiles (e.g., TCS, Infosys, product companies); leaderboard / peer comparison; placement-cell admin dashboard; mentor-review role; behavioral interviews as a separate category from HR

3. Zero-Cost Tech Stack

Layer	Technology	Cost
Frontend	Next.js + React + Tailwind CSS	Free (Vercel free tier hosting)
Backend	Node.js + Express	Free (Render free web service)
Database & Storage	PostgreSQL + file storage via Supabase	Free (Supabase free tier — 500MB DB)
ORM	Prisma	Free, open source
Authentication	Self-rolled JWT (access + refresh tokens)	Free — no third-party auth vendor
AI Engine	Google Gemini API	Free tier quota
Voice (Stretch)	Browser Web Speech API	Free — built into Chrome/Edge
Charts	Recharts	Free, open source
Version Control / CI	GitHub + GitHub Actions	Free for small repos

4. 8-Week Phase Roadmap — Overview

Each phase builds directly on the previous one's deliverable; no phase can start early without its dependency being met.

Wk	Phase	Key Modules Built	Deliverable
1	Foundations	Wireframes for all core screens · Database schema design (Users, Resumes, Interviews, Questions, Responses, Reports) · GitHub repo, branching strategy, project board · Free-tier accounts: Vercel, Render, Supabase, Google AI Studio (Gemini key)	Approved wireframes + ERD + scaffolded repo with CI skeleton
2	Auth & API Foundation	Express project structure + Prisma migrations · Register/login/refresh/logout endpoints (self-rolled JWT) · Role-based access middleware · Login & Signup screens (Next.js + Tailwind) · Staging deploy: Render (backend) + Vercel (frontend)	Working end-to-end login/signup on a live staging URL
3	Interview Flow Skeleton	Interviews / Questions / Responses tables & endpoints · Interview Setup screen (category → role → mode → start) · Live Interview screen: question card, timer, text input, progress bar · Placeholder-question session flow, start to finish	A full mock session can be started and completed with placeholder questions
4	AI Core Working	Isolated AI Service module (generateQuestions, evaluateResponse, generateReport) · Gemini integration for Technical + HR question generation · Gemini integration for response scoring (correctness, communication, structure) · JSON-schema validation of every AI response before the backend trusts it	Live sessions use real AI-generated questions and produce real AI scores

Wk	Phase	Key Modules Built	Deliverable
5	Resume-Aware Interviews	Resume upload to Supabase Storage (PDF/DOCX) · AI resume-parsing prompt → structured JSON (skills, projects, internships, education) · Review/edit screen for the student to correct parsed fields · Wire parsed resume data into the Week 4 question-generation prompt	Resume-specific questions demonstrably appear in a live session
6	Reports & Analytics	Report generation on session completion (summary, strengths, weaknesses, roadmap) · Results/Report screen with question-by-question feedback · History + score-trend view (Recharts line chart) · Stretch: Web Speech API integration for spoken answers	Full report and history populated from real session data; voice input attempted
7	QA Pass	Structured testing: empty resume, AI timeout, abandoned session, slow network · Cross-browser + mobile breakpoint checks · Defect fixes, UI/loading-state polish · Finish voice mode if feasible; otherwise confirm text-mode fallback is fully solid	Stable build with a documented list of known issues, if any
8	Deployment & Submission	Finalize production deployment (Vercel + Render + Supabase) · Production environment variables/secrets set correctly · Final README, API notes, individual contribution summary · Demo script covering the full workflow, rehearsed at least twice with a recorded fallback	Live production deployment + final submission package (code, docs, demo)

5. Detailed Phase Breakdown

Phase 1 — Foundations (Week 1)

Goal: Lock scope and stand up the skeleton every later phase depends on.

Modules / Features Built:

- Wireframes for all core screens
- Database schema design (Users, Resumes, Interviews, Questions, Responses, Reports)
- GitHub repo, branching strategy, project board
- Free-tier accounts: Vercel, Render, Supabase, Google AI Studio (Gemini key)

Deliverable: Approved wireframes + ERD + scaffolded repo with CI skeleton

Dependency on Previous Phase: *None — starting point*

Phase 2 — Auth & API Foundation (Week 2)

Goal: Establish a working, deployed login system so every later feature has a real logged-in user to attach to.

Modules / Features Built:

- Express project structure + Prisma migrations
- Register/login/refresh/logout endpoints (self-rolled JWT)
- Role-based access middleware
- Login & Signup screens (Next.js + Tailwind)
- Staging deploy: Render (backend) + Vercel (frontend)

Deliverable: Working end-to-end login/signup on a live staging URL

Dependency on Previous Phase: *Requires Week 1 schema and repo setup*

Phase 3 — Interview Flow Skeleton (Week 3)

Goal: Prove the full session flow works mechanically before adding real AI, so AI integration in Week 4 has a stable base to plug into.

Modules / Features Built:

- Interviews / Questions / Responses tables & endpoints
- Interview Setup screen (category → role → mode → start)
- Live Interview screen: question card, timer, text input, progress bar
- Placeholder-question session flow, start to finish

Deliverable: A full mock session can be started and completed with placeholder questions

Dependency on Previous Phase: *Requires Week 2 auth (sessions belong to a logged-in user)*

Phase 4 — AI Core Working (Week 4)

Goal: Replace placeholders with real AI so the platform's core value — adaptive questioning and evaluation — is functional.

Modules / Features Built:

- Isolated AI Service module (generateQuestions, evaluateResponse, generateReport)
- Gemini integration for Technical + HR question generation
- Gemini integration for response scoring (correctness, communication, structure)
- JSON-schema validation of every AI response before the backend trusts it

Deliverable: Live sessions use real AI-generated questions and produce real AI scores

Dependency on Previous Phase: *Requires Week 3's session flow to have somewhere to plug AI output into*

Phase 5 — Resume-Aware Interviews (Week 5)

Goal: Personalize the AI questions to each student's actual background — the platform's key differentiator from static question banks.

Modules / Features Built:

- Resume upload to Supabase Storage (PDF/DOCX)
- AI resume-parsing prompt → structured JSON (skills, projects, internships, education)
- Review/edit screen for the student to correct parsed fields
- Wire parsed resume data into the Week 4 question-generation prompt

Deliverable: Resume-specific questions demonstrably appear in a live session

Dependency on Previous Phase: *Requires Week 4's AI question generation pipeline to extend*

Phase 6 — Reports & Analytics (Week 6)

Goal: Close the feedback loop — turn raw scores into something a student can actually act on, and start the voice-mode stretch goal.

Modules / Features Built:

- Report generation on session completion (summary, strengths, weaknesses, roadmap)
- Results/Report screen with question-by-question feedback
- History + score-trend view (Recharts line chart)
- Stretch: Web Speech API integration for spoken answers

Deliverable: Full report and history populated from real session data; voice input attempted

Dependency on Previous Phase: *Requires Week 4 evaluation output and Week 5 session data to summarize*

Phase 7 — QA Pass (Week 7)

Goal: Convert a working prototype into a stable, demo-safe product.

Modules / Features Built:

- Structured testing: empty resume, AI timeout, abandoned session, slow network
- Cross-browser + mobile breakpoint checks
- Defect fixes, UI/loading-state polish
- Finish voice mode if feasible; otherwise confirm text-mode fallback is fully solid

Deliverable: Stable build with a documented list of known issues, if any

Dependency on Previous Phase: *Requires all previous weeks' features to exist so they can be tested together*

Phase 8 — Deployment & Submission (Week 8)

Goal: Ship the final, presentable product and prepare for panel evaluation.

Modules / Features Built:

- Finalize production deployment (Vercel + Render + Supabase)
- Production environment variables/secrets set correctly
- Final README, API notes, individual contribution summary
- Demo script covering the full workflow, rehearsed at least twice with a recorded fallback

Deliverable: Live production deployment + final submission package (code, docs, demo)

Dependency on Previous Phase: *Requires a stable build from Week 7*

6. Notes for the Review Panel

- **Deliberate scope control:** Coding-round editor, company-specific profiles, leaderboard, and admin dashboard are intentionally deferred to keep the 8-week, 3-person, zero-budget plan achievable — not omissions.
- **Zero-cost by design:** Every service used (Vercel, Render, Supabase, Gemini free tier, Web Speech API) has a genuinely free tier sufficient for a pilot; no paid infrastructure is assumed anywhere in this roadmap.
- **Best-effort transparency:** Voice mode is flagged as best-effort, not promised, because browser Speech API reliability varies — text mode is the fully-tested, guaranteed path.